

COMP521 Assignment 1 — Linux System Call and Process Observation Lab

Program Behavior

This program demonstrates how the theoretical concept of processes in operating systems is used in practice on UNIX-like operating systems such as Linux. At the start of the program, the original process obtains and prints its PID using `getpid()`. A child process is then created using the `fork()` system call. The program checks if `fork()` failed by checking if the returned PID is less than 0.

After `fork()` succeeds, both the parent and child continue executing the program. We distinguish between them by checking the value returned by `fork()`. In the child process, the value is 0. In the parent process, the value is the PID assigned to the child process by the OS. The child prints its PID and then uses `exec1()` to execute `/bin/ls`, which is the UNIX command to list contents in a given directory (defaults to current working directory).

The parent process prints its own PID and the child's PID before calling `wait()`. The `wait()` function is used to wait for the child process to finish executing `/bin/ls`. Once it is done, the child process will terminate and the `wait()` function will return.

Evidence Table

OS Concept	C-Level Operation	Actual Trace Evidence	Your Explanation
Process creation	<code>fork()</code>	<code>clone(...) = 6589</code>	Creates child PID 6589.
Program replacement	<code>exec()</code>	<code>execve("/bin/ls", ...) = 0</code>	Child 6589 becomes <code>/bin/ls</code> .
Parent waiting	<code>wait()</code>	<code>wait4(-1 <unfinished ...></code>	Parent 6588 waits for the child.
Output	<code>printf()</code> / <code>ls</code>	<code>write(...) = 89</code>	Child 6589 writes <code>/bin/ls</code> output.

Actual Trace Evidence

```
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|
SIGCHLD, child_tidptr=0x7f4bcc7aaa10) = 6589
[pid 6589] execve("/bin/ls", ["ls"], 0x7fff3f367698 /* 69 vars */) = 0
```

```
[pid 6588] wait4(-1 <unfinished ...>  
<... wait4 resumed>, NULL, 0, NULL)      = 6589  
[pid 6589] write(1, "fork_order_starter.c fork_predi"... , 89) = 89
```

fork() vs. exec()

The `fork()` system call creates a new child process. If `fork()` succeeds, there will be two processes. The child process and its parent. The `exec()` system call replaces the program that is running inside the child process. In this case, the `proc_lab` child process then become an `ls` process since `/bin/ls` is ran using `execl()`. It is not correct to say that `exec()` creates a new process because the new process has already been created using `fork()`. In other words, the child already exists and we are just using `exec()` to replace the program running inside the child process.

Source Code vs. Linux Trace

The C functions used in this program do not always have the same names as the functions used in the `strace` output. The reason for this is because the C functions used are actually library functions that wrap around the core OS system call. For example, `fork()` may appear as `clone()` or another related process-creation call. `execl()` may appear as `execve()`. These differences show how the C-level functions in the source code connect to the actual operations performed by Linux.

Parent/Child Interaction

After `fork()`, both the parent and child continue executing. The child runs `/bin/ls`, while the parent waits for the child using `wait()`. Because the scheduler controls when each process runs, the output before `wait()` can appear in different orders. The parent cannot continue past `wait()` until the child terminates. This provides synchronization and ensures that the parent's final message is printed after the child finishes.

Conclusion

This program demonstrated the connection between C programs and Linux processes. `fork()` creates a child process, `exec()` replaces the program running in that process, and `wait()` allows the parent to synchronize with the child. `strace` showed that these simple C operations correspond to lower-level Linux system calls such as `clone()`, `execve()`, `wait4()`, and `write()`. The experiment connected the theoretical process model to the actual behavior of the Linux operating system.